

JPA + REST v Jave

DETAILNÝ ŠTUDIJNÝ GUIDE NA ZÁPOČET

ORM mapovanie, asociácie, databáza, REST API, JAX-RS, REST + JPA

Tento guide je písaný krok po kroku: najprv UML/model/logika, potom Java entity, potom databázový model, potom detailne čo robí ORM/JPA, typické chyby a praktické použitie.

Obsah

- 0. Mentálny model JPA a ORM
- 1. ManyToOne
- 2. OneToMany
- 3. OneToOne
- 4. ManyToMany
- 5. ElementCollection
- 6. Embedded / Embeddable
- 7. Cascade
- 8. orphanRemoval
- 9. FetchType.LAZY vs EAGER
- 10. JoinColumn
- 11. JoinTable
- 12. mappedBy
- 13. Inheritance stratégie
- 14. REST v Jave
- 15. REST + JPA
- 16. Typické skúškové pasce
- 17. Veľké recap tabuľky

0. Mentálny model JPA a ORM

Java svet	Relačná databáza
-----	-----
@Entity trieda	-----mapuje sa na-----> tabuľka
objekt entity	-----mapuje sa na-----> riadok tabuľky
atribút objektu	-----mapuje sa na-----> stĺpec
@Id	-----mapuje sa na-----> primárny kľúč
referencia na entitu	-----mapuje sa na-----> foreign key
kolekcia entít	-----mapuje sa na-----> FK alebo join tabuľka

JPA je štandard, ktorý umožňuje pracovať s databázou objektovo. Namiesto ručného skladania SQL dotazov vytváraš objekty, meníš ich atribúty a JPA synchronizuje ich stav s databázou. Táto synchronizácia prebieha cez EntityManager a persistence context.

Persistence context je pracovná pamäť JPA. Ak je entita managed, JPA sleduje jej zmeny a pri committe vie vytvoriť SQL INSERT, UPDATE alebo DELETE. Ak je entita detached, existuje ako Java objekt, ale EntityManager ju už nesleduje.

```
@Entity
public class Person {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}

// objekt Person -> riadok v tabuľke PERSON
// id -> primárny kľúč
// name -> stĺpec NAME
```

Pojem	Význam
@Entity	Trieda je perzistentná entita mapovaná na tabuľku.
@Id	Atribút je primárny kľúč.
@GeneratedValue	Kľúč sa generuje automaticky, zvyčajne databázou alebo providerom.
EntityManager	Objekt, cez ktorý robíme persist, find, merge, remove a dotazy.
Owner asociácie	Strana, ktorá reálne zapisuje vzťah do DB - vlastní FK alebo join table.
Inverse side	Opačná strana obojsmernej asociácie, väčšinou používa mappedBy.

1. ManyToOne

1. UML / model / logika

Publisher 1 ----- * Book

Číta sa:

- jeden Publisher má N kníh
- veľa Book riadkov odkazuje na jeden Publisher riadok
- Book * ----- 1 Publisher

2. Java entity triedy

```
import jakarta.persistence.*;

@Entity
public class Publisher {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    public Publisher() {}

    public Publisher(String name) {
        this.name = name;
    }

    // getters and setters
}

@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "publisher_id")
    private Publisher publisher;

    public Book() {}

    public Book(String title, Publisher publisher) {
        this.title = title;
        this.publisher = publisher;
    }

    // getters and setters
}
```

3. Databázový model

```
PUBLISHER
-----
id PK
name

BOOK
-----
id PK
title
publisher_id FK -> PUBLISHER.id
```

4. Detailné vysvetlenie

ManyToOne sa používa vtedy, keď veľa objektov na jednej strane patrí k jednému objektu na druhej strane. V relačnej databáze je to najprirodzenejšie mapovanie: do tabuľky na strane N sa pridá FK stĺpec.

Owner asociácie je Book.publisher, pretože v tabuľke BOOK vzniká stĺpec publisher_id. Owner je vždy strana, ktorá má priamy vplyv na hodnotu FK v databáze.

mappedBy sa tu nepoužíva, pretože ManyToOne je už owner strana. Ak existuje opačná kolekcia Publisher.books, mappedBy bude až tam.

Pri persist: najprv musí existovať Publisher alebo musí byť nastavený cascade. Potom JPA vloží Book s publisher_id. Ak je publisher transient a cascade nie je nastavený, vznikne chyba.

Pri delete knihy sa odstráni riadok BOOK. Pri delete publisheru môže databáza odmietnuť delete, ak na neho stále ukazujú knihy.

Pri fetchovaní má ManyToOne default EAGER, ale v praxi sa často nastavuje LAZY, aby sa zbytočne nenačítal publisher pri každej knihe.

5. Typické chyby

Chyba	Dôsledok / oprava
Nepersistnutý Publisher	Persistuješ Book s novým Publisherom bez cascade. Riešenie: persist publisher alebo nastav cascade vhodne.
FK na zlej strane	FK má byť v BOOK, nie v PUBLISHER.
EAGER všade	Zbytočné SQL a často N+1 problém.
Serializácia entity priamo do JSON	Môže vyvolať lazy loading alebo cykly. Použi DTO.

6. Reálne použitie

- Book -> Publisher
- Order -> Customer
- Comment -> Article
- Employee -> Department

2. OneToMany

1. UML / model / logika

Publisher 1 ----- * Book

Pohľad z opačnej strany:

Publisher má kolekciu Book objektov.

Book stále obsahuje FK publisher_id.

2. Java entity triedy

```
import jakarta.persistence.*;
import java.util.*;
```

```
@Entity
public class Publisher {
```

```
    @Id
    @GeneratedValue
```

```

    private Long id;

    private String name;

    @OneToMany(mappedBy = "publisher")
    private List<Book> books = new ArrayList<>();

    public void addBook(Book book) {
        books.add(book);
        book.setPublisher(this); // důležité: nastaví owner stranu
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.setPublisher(null); // zruší FK na owner strane
    }
}

@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "publisher_id")
    private Publisher publisher;

    public void setPublisher(Publisher publisher) {
        this.publisher = publisher;
    }
}

```

3. Databázový model

```

PUBLISHER
-----
id PK
name

BOOK
-----
id PK
title
publisher_id FK -> PUBLISHER.id

```

Žiadna PUBLISHER_BOOK tabuľka nevzniká, lebo mappedBy hovorí, že FK spravuje Book.publisher.

4. Detailné vysvetlenie

OneToMany je kolekčný pohľad na vzťah. Z objektového hľadiska chceš z publisher-a vidieť jeho knihy. Z databázového hľadiska sa však nič nové nemení: FK stále patrí do tabuľky BOOK.

Owner asociácie nie je Publisher.books. Owner je Book.publisher. Publisher.books je inverse side, pretože používa mappedBy.

mappedBy = "publisher" znamená: Táto kolekcia je zrkadlo atribútu publisher v triede Book. JPA nemá vytvárať ďalší FK ani join tabuľku.

Pri persist je rozhodujúce nastavenie owner strany. Ak zavoláš iba publisher.getBooks().add(book), ale nenastavíš book.setPublisher(publisher), databázový FK nemusí vzniknúť.

Pri delete publisher sa knihy automaticky nemažú, pokiaľ nie je nastavený cascade REMOVE alebo orphanRemoval v kompozícii.

Default fetch pre OneToMany je LAZY, preto sa kolekcia načíta až pri prístupe na books.

5. Typické chyby

Chyba	Dôsledok / oprava
Zabudnuté mappedBy	JPA môže vytvoriť zbytočnú join tabuľku.
Nastavená len inverse side	Pridáš Book do Publisher.books, ale nenastavíš Book.publisher; FK ostane null.
Infinite recursion	Publisher -> books -> publisher -> books pri JSON serializácii.
LazyInitializationException	Kolekciu čítaš po zatvorení EntityManagera.

6. Reálne použitie

- Publisher -> Books
- Customer -> Orders
- Category -> Products
- BlogPost -> Comments

3. OneToOne

1. UML / model / logika

User 1 ----- 1 Profile

Alebo:

Auto 1 ----- 0..1 Insurance

Insurance 1 ----- 1 Auto

2. Java entity triedy

```
import jakarta.persistence.*;

@Entity
public class AppUser {

    @Id
    @GeneratedValue
    private Long id;

    private String username;

    @OneToOne(mappedBy = "user")
    private UserProfile profile;
}

@Entity
public class UserProfile {
```

```

@Id
@GeneratedValue
private Long id;

private String displayName;

@OneToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "user_id", nullable = false, unique = true)
private AppUser user;
}

```

3. Databázový model

```

APP_USER
-----
id PK
username

USER_PROFILE
-----
id PK
display_name
user_id FK UNIQUE NOT NULL -> APP_USER.id

```

4. Detailné vysvetlenie

OneToOne znamená, že jeden riadok jednej tabuľky môže byť spojený najviac s jedným riadkom druhej tabuľky. Technicky sa to často realizuje FK stĺpcom s UNIQUE obmedzením.

Owner je UserProfile.user, pretože tabuľka USER_PROFILE obsahuje user_id. AppUser.profile je inverse side s mappedBy.

nullable = false znamená povinný vzťah: profil bez používateľa nesmie existovať. unique = true zabezpečuje, že jeden používateľ nemôže mať viac profilov.

Pri persist musí byť owner strana nastavená. Nestačí iba user.setProfile(profile), ak profile.user ostane null.

Pri delete treba premyslieť životný cyklus. Ak profil nemá existovať bez usera, používa sa cascade + orphanRemoval alebo sa profil maže ručne.

Default fetch pre OneToOne je EAGER, ale často sa nastavuje LAZY. Pozor: LAZY pri OneToOne nemusí vždy fungovať bez bytecode enhancementu/provider špecifickej podpory.

5. Typické chyby

Chyba	Dôsledok / oprava
Chýba unique constraint	Databáza nevynucuje skutočné 1:1.
mappedBy na oboch stranách	Nevznikne owner, mapovanie je chybné.
JoinColumn na oboch stranách	Vzniknú dve nezávislé FK asociácie.
Zlá povinnosť vzťahu	nullable=true dovolí profil bez usera.

6. Reálne použitie

- User -> Profile
- Auto -> Insurance
- Person -> Passport
- Account -> AccountSettings

4. ManyToMany

1. UML / model / logika

Book * ----- * Author

Číta sa:

- Book má N autorov
- Author má N kníh
- v relačnej DB sa to nedá uložiť jedným FK stĺpcom
- treba join tabuľku

2. Java entity triedy

```
import jakarta.persistence.*;
import java.util.*;

@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ManyToMany
    @JoinTable(
        name = "book_author",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "author_id")
    )
    private List<Author> authors = new ArrayList<>();
}

@Entity
public class Author {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToMany(mappedBy = "authors")
    private List<Book> books = new ArrayList<>();
}
```

3. Databázový model

BOOK

id PK

title

AUTHOR

id PK

name


```
BOOK_AUTHOR
-----
book_id FK -> BOOK.id
author_id FK -> AUTHOR.id
PK(book_id, author_id)
```

4. Detailné vysvetlenie

ManyToMany potrebuje join tabuľku, pretože jeden FK stĺpec nevie vyjadriť vzťah N:N. Join tabuľka obsahuje dvojice ID.

Owner je Book.authors, pretože tam je @JoinTable. Author.books je inverse side s mappedBy = "authors".

mappedBy pri ManyToMany znamená: druhá strana nevytvára vlastnú join tabuľku, ale používa join tabuľku definovanú na owner strane.

Pri persist bez cascade treba persistnúť Book aj všetkých Author objektov. Potom JPA vloží záznamy do BOOK_AUTHOR.

Pri delete knihy JPA najprv odstráni riadky z BOOK_AUTHOR pre danú knihu a potom zmaže BOOK. Autorov nemaže, pokiaľ nie je nevhodne nastavený cascade REMOVE.

Default fetch pre ManyToMany je LAZY. To je správne, lebo N:N kolekcie môžu byť veľké.

5. Typické chyby

Chyba	Dôsledok / oprava
CascadeType.REMOVE na ManyToMany	Nebezpečné: zmazanie knihy by mohlo zmazať autora používaného inými knihami.
Zabudnuté mappedBy	Môžu vzniknúť dve join tabuľky alebo nekonzistentné mapovanie.
Duplicítne dvojice	Použi Set alebo unique constraint/PK v join tabuľke.
Priamy JSON export	Cyklické referencie Book -> Author -> Book.

6. Reálne použitie

- Books <-> Authors
- Students <-> Courses
- Users <-> Roles
- Products <-> Tags

5. ElementCollection

1. UML / model / logika

```
Book 1 ----- * String
```

Book má kolekciu jednoduchých hodnôt:

- authors ako List<String>
- phoneNumbers ako List<String>
- tags ako Set<String>

Nie sú to entity, nemajú vlastné @Id.

2. Java entity triedy

```
import jakarta.persistence.*;
import java.util.*;

@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    private String title;

    @ElementCollection
    @CollectionTable(
        name = "book_tag",
        joinColumns = @JoinColumn(name = "book_id")
    )
    @Column(name = "tag")
    private Set<String> tags = new HashSet<>();
}
```

3. Databázový model

BOOK

id PK

title

BOOK_TAG

book_id FK -> BOOK.id

tag

BOOK_TAG nemá samostatnú entitu Tag.

Riadky v BOOK_TAG existujú iba ako hodnoty kolekcie entity Book.

4. Detailné vysvetlenie

ElementCollection sa používa pre kolekcie jednoduchých hodnôt alebo embeddable objektov. Prvky kolekcie nie sú entity, nemajú identitu, nemajú vlastný lifecycle mimo vlastníka.

Owner je entita Book. Kolekčná tabuľka BOOK_TAG je vlastnená cez FK book_id.

mappedBy sa pri ElementCollection nepoužíva, lebo nejde o asociáciu medzi dvoma entitami.

Pri persist knihy JPA vloží BOOK a potom riadky do BOOK_TAG. Pri zmene kolekcie môže provider vymazať a znovu vložiť riadky kolekcie.

Pri delete knihy sa zmažú aj jej element collection riadky, pretože nemajú samostatnú existenciu.

Default fetch pre ElementCollection je LAZY.

5. Typické chyby

Chyba	Dôsledok / oprava
Použitie pre entity	Ak má prvok vlastné ID a životný cyklus, má to byť @OneToMany, nie ElementCollection.
Očakávanie samostatného repository	ElementCollection nie je entita.

Veľké kolekcie	Môžu byť neefektívne pri update.
Zabudnutý @CollectionTable	JPA vygeneruje názvy automaticky, často nie podľa očakávania.

6. Reálne použitie

- Book -> tags
- User -> phoneNumbers
- Product -> colors
- Order -> simple notes

6. Embedded / Embeddable

1. UML / model / logika

Customer 1 ----- 1 Address

Ale pozor:

Address nie je samostatná entita.

Address je hodnota vložená do tabuľky Customer.

Customer

- id
- name
- address.street
- address.city

2. Java entity triedy

```
import jakarta.persistence.*;

@Embeddable
public class Address {

    private String street;
    private String city;
    private String zipCode;
}

@Entity
public class Customer {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @Embedded
    private Address address;
}
```

3. Databázový model

CUSTOMER

id PK

```
name
street
city
zip_code
```

Žiadna ADDRESS tabuľka nevzniká.
Address nemá vlastné ID.

4. Detailné vysvetlenie

Embeddable je vložený hodnotový objekt. Používa sa, keď chceš v Java modeli peknú triedu, ale v databáze nechceš samostatnú tabuľku.

Owner je entita, ktorá obsahuje @Embedded atribút. FK nevzniká, lebo nejde o asociáciu medzi tabuľkami.

mappedBy sa nepoužíva. Embedded objekt nie je druhá entita.

Pri persist Customer sa stĺpce Address vložia priamo do riadku CUSTOMER.

Pri delete Customer sa nič osobitné nemaže, lebo Address nemá vlastný riadok.

Fetchovanie sa rieši ako pri asociáciách. Embedded hodnoty sú súčasťou riadku entity.

5. Typické chyby

Chyba	Dôsledok / oprava
Použitie @Entity namiesto @Embeddable	Vznikne samostatná tabuľka a zbytočný vzťah.
Očakávanie vlastného ID	Embeddable nemá identitu.
Duplicitné názvy stĺpcov	Pri viacerých embedded objektoch treba @AttributeOverride.
Zdieľanie embedded objektu medzi entitami	Hodnotový objekt sa má chápať ako súčasť vlastníka.

6. Reálne použitie

- Customer -> Address
- Order -> Money
- Person -> ContactInfo
- Insurance -> embedded car data

7. Cascade

1. UML / model / logika

```
Order 1 ----- 1..* OrderItem
```

Order je agregát/root.

OrderItem bez Order nedáva zmysel.

Preto niektoré operácie môžu kaskádovať z Order na OrderItem.

2. Java entity triedy

```
import jakarta.persistence.*;
import java.util.*;

@Entity
public class CustomerOrder {

    @Id
```

```

@GeneratedValue
private Long id;

@OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
private List<OrderItem> items = new ArrayList<>();

public void addItem(OrderItem item) {
    items.add(item);
    item.setOrder(this);
}
}

@Entity
public class OrderItem {

    @Id
    @GeneratedValue
    private Long id;

    private String productName;
    private int quantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id")
    private CustomerOrder order;

    public void setOrder(CustomerOrder order) {
        this.order = order;
    }
}

```

3. Databázový model

```

CUSTOMER_ORDER
-----
id PK

ORDER_ITEM
-----
id PK
product_name
quantity
order_id FK -> CUSTOMER_ORDER.id

```

4. Detailné vysvetlenie

Cascade nehovorí, kde vznikne FK. FK stále určuje asociácia a owner strana. Cascade hovorí, či sa operácia vykonaná na rodičovi prenesie aj na asociované objekty.

Cascade typ	Význam
PERSIST	Keď persistneš parent, persistnú sa aj children.
MERGE	Keď mergeš parent, mergnú sa aj children.
REMOVE	Keď zmažeš parent, zmažú sa aj children.
REFRESH	Refresh parent sa prenesie aj na children.
DETACH	Detach parent sa prenesie aj na children.
ALL	Všetko vyššie.

Pri persist s CascadeType.ALL stačí zavolať `em.persist(order)`. JPA prejde graf objektov a vloží aj `OrderItem`. Ale FK `order_id` sa správne nastaví len vtedy, keď je nastavená owner strana `item.order`.

Cascade používaj len tam, kde je jasný vlastnícky životný cyklus. Nepoužívaj CascadeType.REMOVE na ManyToMany medzi zdieľanými entitami, napríklad Book a Author.

5. Typické chyby

Chyba	Dôsledok / oprava
Cascade si mylíš s FK	Cascade nerieši databázové stĺpce, iba šírenie operácií.
Cascade REMOVE na zdieľané entity	Môžeš zmazať dáta používané inde.
Nastavená len inverse side	Cascade persistne child, ale FK môže byť null.
Cascade ALL všade	Nečakané mazanie a veľké operácie.

6. Reálne použitie

- Order -> OrderItem
- ShoppingCart -> CartItem
- Invoice -> InvoiceLine
- BlogPost -> Comments v jednoduchých systémoch

8. orphanRemoval

1. UML / model / logika

ShoppingCart 1 ----- * CartItem

Kompozícia:

CartItem existuje len ako súčasť ShoppingCart.

Ak položku odstránim z košíka, má zmiznúť aj z DB.

2. Java entity triedy

```
@Entity
public class ShoppingCart {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(
        mappedBy = "cart",
        cascade = CascadeType.ALL,
        orphanRemoval = true
    )
    private List<CartItem> items = new ArrayList<>();

    public void addItem(CartItem item) {
        items.add(item);
        item.setCart(this);
    }

    public void removeItem(CartItem item) {
        items.remove(item);
        item.setCart(null);
    }
}

@Entity
public class CartItem {
```

```

@Id
@GeneratedValue
private Long id;

private String productName;
private int quantity;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "cart_id")
private ShoppingCart cart;

public void setCart(ShoppingCart cart) {
    this.cart = cart;
}
}

```

3. Databázový model

```

SHOPPING_CART
-----
id PK

CART_ITEM
-----
id PK
product_name
quantity
cart_id FK -> SHOPPING_CART.id

```

4. Detailné vysvetlenie

orphanRemoval = true znamená: ak child objekt prestane patriť parentovi, stáva sa sirotou a JPA ho odstráni z databázy. Nestačí teda len nastaviť FK na null.

Rozdiel oproti CascadeType.REMOVE: Cascade REMOVE sa spustí, keď zmažeš parent. orphanRemoval sa spustí, keď odstrániš child z kolekcie parenta.

V materiáloch je tento prípad vysvetlený ako kompozícia nákupného košíka a položiek. Samotné CascadeType.REMOVE nestačí na zmazanie položky odstránenej zo zoznamu; orphanRemoval zabezpečí zmazanie záznamu z tabuľky položiek.

Owner FK je CartItem.cart, pretože FK cart_id je v tabuľke CART_ITEM. orphanRemoval je ale nastavené na kolekciu parenta, lebo opisuje životný cyklus dieťaťa v rámci rodiča.

5. Typické chyby

Chyba	Dôsledok / oprava
Používanie orphanRemoval na zdieľané deti	Ak child môže patriť aj inde, nemaž ho automaticky.
Myslieť si, že Cascade REMOVE stačí	Nestačí pri odstránení child z kolekcie.
Nenastavená owner strana na null	Pomocné metódy musia synchronizovať obe strany.
Používanie pri ManyToMany	ManyToMany nemá typický parent-child lifecycle.

6. Reálne použitie

- ShoppingCart -> CartItem
- Order -> OrderLine
- Form -> FormField
- Document -> Paragraph v kompozičnom modeli

9. FetchType.LAZY vs EAGER

1. UML / model / logika

```
Book * ----- 1 Publisher
Publisher 1 ----- * Book
Book * ----- * Author
```

Otázka fetchovania:

Keď načítam Book, načíta sa hneď aj Publisher?

Keď načítam Publisher, načítajú sa hneď všetky Books?

2. Java entity triedy

```
@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "publisher_id")
    private Publisher publisher;

    @ManyToMany(fetch = FetchType.LAZY)
    private List<Author> authors = new ArrayList<>();
}

@Entity
public class Publisher {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "publisher", fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();
}
```

3. Databázový model

FetchType nemení databázový model.

Mení iba to, kedy JPA vykoná SELECT dotazy.

BOOK.publisher_id FK stále existuje.

BOOK_AUTHOR join tabuľka stále existuje.

4. Detailné vysvetlenie

EAGER znamená: JPA sa snaží asociáciu načítať hneď spolu s entitou. LAZY znamená: JPA načíta asociáciu až vtedy, keď na ňu pristúpiš.

Asociácia	Default fetch
@ManyToOne	EAGER
@OneToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY
@ElementCollection	LAZY

LAZY je väčšinou bezpečnejšia voľba pre výkon. Problém vzniká, ak sa pokúsiš načítať LAZY asociáciu mimo otvoreného persistence contextu. Vtedy môže vzniknúť LazyInitializationException.

Správne riešenie nie je dať všetko na EAGER. Správne riešenie je načítať potrebné dáta v servisnej vrstve pomocou JOIN FETCH, EntityGraph alebo DTO projekcie.

5. Typické chyby

Chyba	Dôsledok / oprava
Všetko EAGER	Veľa zbytočných dotazov, veľké grafy objektov.
Čítanie LAZY po zatvorení transakcie	LazyInitializationException.
N+1 problém	Pre 100 kníh sa načíta 100 publisherov samostatne. Použi JOIN FETCH.
Entity priamo v REST odpovedi	Serializátor spúšťa lazy loading mimo kontroly.

6. Reálne použitie

- LAZY pre väčšinu kolekcí
- LAZY pre ManyToOne vo väčších aplikáciách
- EAGER len keď je asociácia malá a vždy potrebná

10. JoinColumn

1. UML / model / logika

```
Book * ----- 1 Publisher
```

JoinColumn definuje FK stĺpec:
BOOK.publisher_id -> PUBLISHER.id

2. Java entity triedy

```
@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(
        name = "publisher_id",
        nullable = false,
        referencedColumnName = "id"
    )
    private Publisher publisher;
}
```

3. Databázový model

```
BOOK
-----
id PK
title
publisher_id FK NOT NULL -> PUBLISHER.id
```

4. Detailné vysvetlenie

@JoinColumn sa používa na strane, kde vzniká FK stĺpec. Najčastejšie pri ManyToOne a OneToOne owner strane.

name je názov FK stĺpca v aktuálnej tabuľke. referencedColumnName je stĺpec v cieľovej tabuľke, typicky id.

nullable = false vynucuje povinnú asociáciu. unique = true sa používa pri OneToOne.

JoinColumn nie je to isté ako mappedBy. JoinColumn hovorí o databázovom stĺpci. mappedBy hovorí o Java atribúte na owner strane.

Pri persist hodnota JoinColumn vznikne z ID objektu, na ktorý referencia ukazuje.

Pri delete sa databáza riadi FK constraintom. Ak na parent ukazujú children, delete parenta môže zlyhať.

5. Typické chyby

Chyba	Dôsledok / oprava
JoinColumn na inverse strane	Inverse strana nemá zapisovať FK.
mappedBy = názov stĺpca	mappedBy má byť názov Java atribútu, nie FK stĺpca.
nullable nesedí s UML	Ak UML hovorí 1, FK má byť NOT NULL.
Chýba unique pri 1:1	DB dovolí viac záznamov na jednu entitu.

6. Reálne použitie

- Book.publisher_id
- Order.customer_id
- Profile.user_id
- Comment.article_id

11. JoinTable

1. UML / model / logika

Book * ----- * Author

JoinTable je samostatná prepojovacia tabuľka:
BOOK_AUTHOR(book_id, author_id)

2. Java entity triedy

```
@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToMany
    @JoinTable(
        name = "book_author",
        joinColumns = @JoinColumn(name = "book_id"),
        inverseJoinColumns = @JoinColumn(name = "author_id")
    )
    private Set<Author> authors = new HashSet<>();
}
```

3. Databázový model

```
BOOK
-----
id PK

AUTHOR
-----
id PK

BOOK_AUTHOR
-----
book_id FK -> BOOK.id
author_id FK -> AUTHOR.id
PK(book_id, author_id)
```

4. Detailné vysvetlenie

@JoinTable definuje prepojavaciu tabuľku. Používa sa hlavne pri ManyToMany, ale môže sa objaviť aj pri jednosmernom OneToMany, ak nechceš FK v child tabuľke alebo provider tak mapuje defaultne.

joinColumns opisuje FK na owner entitu. inverseJoinColumns opisuje FK na druhú stranu asociácie.

Owner je strana, kde je @JoinTable. Táto strana rozhoduje o riadkoch v join tabuľke.

Pri persist JPA vloží entity a potom vloží riadky do join tabuľky. Pri odstránení väzby z kolekcie sa maže riadok z join tabuľky, nie nevyhnutne samotný Author.

Pri delete Book sa odstránia súvisiace riadky v BOOK_AUTHOR a potom Book. Author ostáva, ak nie je nastavený nebezpečný cascade REMOVE.

Fetchovanie ManyToMany je default LAZY.

5. Typické chyby

Chyba	Dôsledok / oprava
Zamieňanie joinColumns a inverseJoinColumns	Vzniknú stĺpce s opačným významom.
Chýba unique/PK	Duplicitné väzby.
Cascade REMOVE	Môže zmazať zdieľané entity.
JoinTable tam, kde stačí FK	Zbytočná tabuľka, horší model.

6. Reálne použitie

- Book <-> Author
- Student <-> Course
- User <-> Role
- Product <-> Tag

12. mappedBy - najdôležitejšia skúšková téma

1. UML / model / logika

```
Publisher 1 ----- * Book
```

V Java modeli sú dve referencie:

```
Publisher.books -----> List<Book>
Book.publisher -----> Publisher
```

Ale v databáze je iba jeden vzťah:
BOOK.publisher_id FK -> PUBLISHER.id

2. Java entity triedy

```
@Entity
public class Publisher {

    @Id
    @GeneratedValue
    private Long id;

    @OneToMany(mappedBy = "publisher")
    private List<Book> books = new ArrayList<>();
}

@Entity
public class Book {

    @Id
    @GeneratedValue
    private Long id;

    @ManyToOne
    @JoinColumn(name = "publisher_id")
    private Publisher publisher;
}
```

3. Databázový model

SPRÁVNE:

PUBLISHER

id PK

BOOK

id PK

publisher_id FK

NESPRÁVNE pri zabudnutom mappedBy:

PUBLISHER

BOOK

PUBLISHER_BOOK <-- zbytočná join tabuľka alebo druhá asociácia

4. Detailné vysvetlenie

mappedBy sa používa iba pri obojsmerných asociáciách. Obojsmerná asociácia znamená, že v Java modeli sa vieš navigovať oboma smermi: z Book na Publisher aj z Publisher na Book.

Databáza však nepotrebuje dve väzby. Potrebuje jeden FK alebo jednu join tabuľku. Preto JPA musí vedieť, ktorá Java referencia je owner a ktorá je len inverse side.

mappedBy píšeš na inverse side. Hodnota mappedBy je názov Java atribútu na owner strane. Nie je to názov tabuľky. Nie je to názov FK stĺpca.

V príklade mappedBy = "publisher" znamená: kolekcia Publisher.books je namapovaná atribútom Book.publisher. Owner je Book.publisher. FK je BOOK.publisher_id.

Ak nastavíš iba inverse side, napríklad publisher.getBooks().add(book), JPA nemusí zapísať FK. Rozhodujúca je owner strana: book.setPublisher(publisher).

Pri ManyToMany môže byť owner hociktorá strana, ale iba jedna. Druhá strana musí mať mappedBy. Owner definuje JoinTable.

5. Typické chyby

Chyba	Správne chápanie
mappedBy = "publisher_id"	Zle. Má byť Java atribút: mappedBy = "publisher".
mappedBy na owner strane	Zle. mappedBy je len na inverse strane.
mappedBy na oboch stranách	Zle. Potom nikto nevlastní asociáciu.
Žiadne mappedBy v obojsmernej väzbe	JPA môže vytvoriť dve nezávislé asociácie.
Nastavená len kolekcia	FK sa nemusí aktualizovať. Nastav owner stranu.

6. Reálne použitie

- Publisher.books mappedBy Book.publisher
- Author.books mappedBy Book.authors
- User.profile mappedBy Profile.user
- Department.employees mappedBy Employee.department

13. Inheritance stratégie

1. UML / model / logika

Element

```
|
|-- TextElement
|
|-- RectElement
```

Element má spoločné atribúty:

- id
- name
- xpos
- ypos

TextElement má navyše:

- text
- fontSize
- font

RectElement má navyše:

- width
- height

2. Java entity triedy

```
import jakarta.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public class Element {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
    private int xpos;
    private int ypos;
}

@Entity
public class TextElement extends Element {

    private String text;
    private int fontSize;
    private String font;
}

@Entity
public class RectElement extends Element {

    private int width;
    private int height;
}
```

3. Databázový model

SINGLE_TABLE:

ELEMENT

id PK

DTYPE

name

xpos

ypos

text

font_size

font

width

height

JOINED:

ELEMENT(id PK, DTYPE, name, xpos, ypos)

TEXT_ELEMENT(id PK/FK -> ELEMENT.id, text, font_size, font)

RECT_ELEMENT(id PK/FK -> ELEMENT.id, width, height)

TABLE_PER_CLASS:

TEXT_ELEMENT(id PK, name, xpos, ypos, text, font_size, font)

RECT_ELEMENT(id PK, name, xpos, ypos, width, height)

4. Detailné vysvetlenie

Stratégia	Ako vyzerá DB	Výhody	Nevýhody
SINGLE_TABLE	Jedna tabuľka pre celú hierarchiu + DTYPE.	Rýchle polymorfné dotazy, jednoduché.	Veľa NULL stĺpcov pre atribúty podtried.
JOINED	Parent tabuľka + child tabuľky spojené cez PK/FK.	Normalizované, menej NULL.	Dotazy často potrebujú JOIN.
TABLE_PER_CLASS	Každá konkrétna trieda má vlastnú tabuľku.	Jednoduché find pri známej triede.	Polymorfné dotazy musia prehľadávať viac tabuliek.

DTYPE je diskriminátorový stĺpec, ktorý hovorí, aký konkrétny typ objektu je uložený v riadku. Pri SINGLE_TABLE a často aj JOINED pomáha JPA zistiť, či má vytvoriť TextElement alebo RectElement.

Pri @MappedSuperclass sa rodičovská trieda nemapuje ako samostatná entita. Jej atribúty sa iba zdedia do tabuliek potomkov. Používa sa, keď nechceš dotazovať parent typ samostatne.

5. Typické chyby

Chyba	Dôsledok / oprava
SINGLE_TABLE s mnohými podtriedami	Tabuľka má veľa NULL stĺpcov.
JOINED pri masívnom čítaní	Veľa JOIN dotazov.
TABLE_PER_CLASS pri polymorfných dotazoch	Zložité UNION/dotazy cez viac tabuliek.
Zamieňanie @MappedSuperclass a @Entity	@MappedSuperclass nie je samostatne dotazovateľná entita.

6. Reálne použitie

- Payment -> CardPayment / BankTransferPayment
- Content -> Article / Video / Podcast
- Element -> TextElement / RectElement
- User -> AdminUser / CustomerUser

14. REST v Jave

14.1 Základný model REST

```
Klient ----HTTP request----> REST server
Klient <--HTTP response--- REST server

Resource = zdroj identifikovaný URI
Representation = JSON / XML / text / HTML / súbor

Príklad URI:
/resources/books
/resources/books/10
/resources/books/10/authors
```

REST je architektonický štýl pre komunikáciu klient-server. Server je bezstavový: neuchováva stav klienta medzi requestmi. Každý request musí obsahovať všetko potrebné na jeho spracovanie.

HTTP metóda	Význam pri kolekcii	Význam pri konkrétnom zdroji
GET	Vráti zoznam alebo odkazy na zdroje.	Vráti reprezentáciu konkrétneho zdroja.
POST	Vytvorí nový zdroj v kolekcii; URI často určí server.	Menej typické; môže vytvoriť podzdroj.
PUT	Nahradí kolekciu, menej používané.	Vytvorí alebo nahradí konkrétny zdroj; idempotentné.
DELETE	Zmaže kolekciu, menej používané.	Zmaže konkrétny zdroj.

14.2 JAX-RS trieda zdroja

```
import jakarta.inject.Singleton;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.MediaType;
import java.util.*;

@Singleton
@Path("menu")
public class MenuResource {

    private final List<String> meals = new ArrayList<>();

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String countMeals() {
        return String.valueOf(meals.size());
    }

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    public void addMeal(String meal) {
        meals.add(meal);
    }

    @GET
    @Path("{index}")
    @Produces(MediaType.TEXT_PLAIN)
    public String getMeal(@PathParam("index") int index) {
        if (index < 0 || index >= meals.size()) {
            throw new NotFoundException();
        }
        return meals.get(index);
    }

    @DELETE
    @Path("{index}")
    public void deleteMeal(@PathParam("index") int index) {
        if (index < 0 || index >= meals.size()) {
            throw new NotFoundException();
        }
        meals.remove(index);
    }
}
```

@Path na triede určuje koreň URI zdroja. @GET, @POST, @PUT, @DELETE mapujú HTTP metódy na Java metódy. @Produces hovorí, aký formát odpoveď vracia. @Consumes hovorí, aký formát request telo prijíma. @PathParam vyberá hodnotu z URI.

14.3 POST vs PUT

Situácia	Použi	Prečo
Server generuje ID/URI	POST /books	Klient pošle dáta, server vytvorí zdroj a určí ID.
Klient pozná ID/URI	PUT /books/ISBN123	Klient ukladá reprezentáciu na konkrétne URI.
Čiastočná úprava	PATCH	Nie je v základných príkladoch, ale v praxi bežné.
Vymazanie	DELETE /books/10	Odstráni konkrétny zdroj.


```

@POST
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
public Response createBook(BookDto dto, @Context UriInfo uriInfo) {
    Long id = service.create(dto);

    URI location = uriInfo.getAbsolutePathBuilder()
        .path(String.valueOf(id))
        .build();

    return Response.created(location).build();
}

@PUT
@Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON)
public Response replaceBook(@PathParam("id") Long id, BookDto dto) {
    service.replace(id, dto);
    return Response.noContent().build();
}

```

15. REST + JPA

15.1 Typická architektúra



REST trieda by nemala obsahovať všetku databázovú logiku. Čistejší model je rozdeliť aplikáciu na resource, service a persistence vrstvu. Resource rieši HTTP, service rieši pravidlá, EntityManager rieši JPA.

15.2 Entity vs DTO

Prístup	Výhody	Nevýhody
Vracanie entít priamo	Rýchle v malých demo projektoch.	Cykly, LazyInitializationException, únik interných polí, problém s JSON/XML.
DTO	Kontroluješ presný tvar API, žiadne cykly, bezpečnejšie.	Viac kódu na mapovanie.

```

public class BookDto {
    public Long id;
    public String title;
    public Long publisherId;
    public String publisherName;
}

@GET

```

```

@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public BookDto getBook(@PathParam("id") Long id) {
    Book book = em.find(Book.class, id);
    if (book == null) {
        throw new NotFoundException();
    }

    BookDto dto = new BookDto();
    dto.id = book.getId();
    dto.title = book.getTitle();
    dto.publisherId = book.getPublisher().getId();
    dto.publisherName = book.getPublisher().getName();
    return dto;
}

```

15.3 Cyklické referencie

Book -> authors -> books -> authors -> books -> ...

Publisher -> books -> publisher -> books -> ...

Toto je problém pri JSON/XML serializácii entít.

Pri obojsmerných JPA asociáciách vie objektový graf obsahovať cykly. Serializátor JSON/XML potom môže prechádzať donekonečna alebo načítať obrovské množstvo dát.

Riešenia: používať DTO, posielat iba ID asociovaných zdrojov, alebo používať HATEOAS - teda linky na súvisiace zdroje namiesto vkladania celých objektov.

15.4 HATEOAS

GET /bookstores/abc

Response body:

```

{
  "name": "Svet kníh"
}

```

HTTP Links:

```

</books/978123>; rel="offer"
</books/978456>; rel="offer"

```

HATEOAS znamená Hypermedia as the Engine of Application State. Namiesto toho, aby server vložil celé asociované objekty do odpovede, pošle klientovi odkazy na ďalšie zdroje. Tým sa znižuje problém cyklických referencií.

```

@GET
@Path("/{id}")
@Produces(MediaType.APPLICATION_JSON)
public Response getBookstore(@Context UriInfo uriInfo,
                             @PathParam("id") Long id) {
    Bookstore store = em.find(Bookstore.class, id);
    if (store == null) {
        throw new NotFoundException();
    }

    Response.ResponseBuilder response = Response.ok(toDto(store));

    for (Book book : store.getBooks()) {

```

```

        response.link(
            uriInfo.getBaseUriBuilder()
                .path("books")
                .path(book.getId().toString())
                .build(),
            "book"
        );
    }

    return response.build();
}

```

16. Typické skúškové pasce

Pasca	Správna odpoveď
Kde vzniká FK pri 1:N?	Na strane N, teda v tabuľke child entity.
Čo je owner?	Strana, ktorá zapisuje FK alebo join table.
Čo je inverse side?	Opačná strana obojsmernej asociácie; typicky má mappedBy.
Čo obsahuje mappedBy?	Názov Java atribútu na owner strane, nie názov DB stĺpca.
Kedy vzniká join table?	Pri ManyToMany, ElementCollection a niekedy pri jednosmernom OneToMany.
Cascade vs orphanRemoval	Cascade šíri operáciu; orphanRemoval maže child po odstránení zo vzťahu.
LAZY vs EAGER	LAZY načíta neskôr, EAGER hneď. FetchType nemení DB model.
REST POST vs PUT	POST na kolekciu, URI určí server. PUT na konkrétne URI, URI pozná klient.
Entity v REST odpovedi	Môžu spôsobiť cykly a lazy loading; lepšie DTO/HATEOAS.

17. Veľké recap tabuľky

17.1 Recap podľa anotácií

Anotácia	Kedy sa používa	Kde vzniká FK	Owner	Typický use-case
@ManyToOne	Veľa entít patrí jednej entite.	V tabuľke na strane Many.	Strana s @ManyToOne.	Book -> Publisher, Order -> Customer
@OneToMany	Jedna entita má kolekciu child entít.	Zvyčajne v child tabuľke na strane Many.	Nie kolekcia, ale opačný @ManyToOne.	Publisher -> Books
@OneToOne	Jedna entita je spojená s najviac jednou druhou entitou.	Na owner strane s @JoinColumn.	Strana s FK.	User -> Profile
@ManyToMany	Obe strany majú viacero objektov.	Nevzniká v entity tabuľke; vzniká join table.	Strana s @JoinTable.	Book <-> Author
@ElementCollection	Kolekcia jednoduchých hodnôt alebo embeddables.	V collection table je FK na owner entitu.	Entita, ktorá vlastní kolekciu.	Book -> tags
@Embedded	Vložený hodnotový objekt v entite.	Nevzniká FK.	Entita s @Embedded.	Customer -> Address
@JoinColumn	Definuje FK stĺpec.	V aktuálnej tabuľke.	Strana s @JoinColumn.	publisher_id, user_id
@JoinTable	Definuje prepojovaciu tabuľku.	V join tabuľke sú dva FK.	Strana s @JoinTable.	book_author
mappedBy	Označuje inverse side obojsmernej asociácie.	Nevytvára FK.	Owner je atribút uvedený v mappedBy.	Publisher.books mappedBy Book.publisher
cascade	Šírenie persist/remove/merge operácií.	Nemení FK.	Neurčuje ownera.	Order -> Items
orphanRemoval	Zmazanie child po odpojení od parenta.	Nemení FK.	Používa sa na parent kolekciu.	Cart -> Items

@Inheritance	Mapovanie dedičnosti entít.	Podľa stratégie.	Koreň hierarchie.	Element -> TextElement
--------------	-----------------------------	------------------	-------------------	------------------------

17.2 Recap podľa asociácií

Asociácia	Potrebuje mappedBy?	Vzniká join tabuľka?	Default fetch
ManyToOne	Nie.	Nie, vzniká FK v Many tabuľke.	EAGER
OneToMany bidirectional	Áno, na One strane.	Nie, ak je mappedBy správne.	LAZY
OneToMany unidirectional	Nie.	Môže vzniknúť join tabuľka alebo FK podľa mapovania/provideru.	LAZY
OneToOne bidirectional	Áno, na inverse strane.	Nie, zvyčajne FK + UNIQUE.	EAGER
ManyToMany bidirectional	Áno, na jednej strane.	Áno.	LAZY
ElementCollection	Nie.	Áno, collection table.	LAZY
Embedded	Nie.	Nie.	Súčasť entity

18. Ako rozmýšľať pri každej úlohe na zápočte

1. Najprv nakresli UML kardinality: 1, N, *, 0..1, 1..*.
2. Urči, či ide o entitu alebo iba hodnotový objekt. Ak má vlastnú identitu, je to entita. Ak nie, môže byť Embedded alebo ElementCollection.
3. Urči owner stranu: kto má FK alebo kto vlastní join tabuľku.
4. Ak je asociácia obojsmerná, na inverse strane napíš mappedBy s názvom Java atribútu owner strany.
5. Nakresli databázové tabuľky a skontroluj, kde je FK.
6. Premysli persist: ktoré objekty musia byť managed alebo cascade persistnuté.
7. Premysli delete: či sa majú mazať iba väzby, child záznamy alebo aj zdieľané entity.
8. Premysli fetch: či nehrozí LazyInitializationException alebo N+1 problém.
9. Pri REST nikdy slepo nevracaj celý JPA graf entít. Použi DTO alebo linky.

Poznámka k materiálom

Guide je zostavený podľa priložených materiálov k JPA a REST: úvod do JPA, entity a anotácie, jednosmerné a obojsmerné asociácie, kompozícia, 1:1, embedded, dedičnosť, REST, REST + JPA, cyklické referencie a HATEOAS.